Angewandte Computer- und Biowissenschaften

Konzept

HOCHSCHULE MITTWEIDA
UNIVERSITY OF APPLIED SCIENCES

F2: Suche von Zeichenketten in Texten – geführte Simulation

Autoren:
Alina Meißner, Madlien Apfelstädt, Jan Hoffmann, Shayan von Düring, Theodor Tran, Tom Spitzbarth, Michael Zwarg, Jacob Friedrichsdorf

Modul:
Algorithmen und Datenstrukturen

Dozent:
Dr. rer. Nat. Rico Beier-Grunwald

Einreichung:
Mittweida, 26.10.22

Inhalt

Inhalt ..... **I**

1 Einführung ..... Fehler! Textmarke nicht definiert.

2 Algorithmen **2**

2.1 Naive Suche *2*

2.2 Boyer-Moore *2*

2.3 Knuth-Morris-Pratt *4*

2.4 Suffix-Baum *5*

3 Didaktisches Konzept **6**

3.1 Zielgruppe *6*

3.2 Lehr- und Lernziele *6*

3.3 Didaktik *6*

3.4 Umsetzung *6*

3.5 Begründung der Entscheidungen *7*

4 Visuelles Konzept **8**

4.1 Designentscheidungen zur Visualisierung und Interaktion *8*

4.2 Begründung der Entscheidungen *9*

5 Verweise **10**

1 Einführung

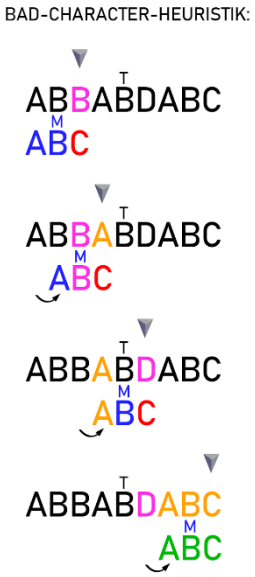
Nach Veröffentlichung des ersten, serienmäßig produzierbaren Mikroprozessors in den 1970er Jahren und der damit folgenden Entwicklung hin zum modernen Computer, musste sich die noch junge Wissenschaft der Informatik mit einem neuartigen Problem auseinandersetzen. Einhergehend mit der digitalen Speicherung von Daten benötigte man nun Verfahren zur Suche und Analyse von Zeichenketten in digitalen Texten. Schnell erarbeitete man eine Vielzahl naiver Ansätze, welche allerdings den steigenden Geschwindigkeitsanforderungen nicht gerecht kamen und deshalb durch effizientere Lösungen ersetzt wurden. Heute zählen unter anderem die Algorithmen von Boyer-Moore, Knuth-Morris-Pratt und der sogenannte Suffix-Baum zu den schnellsten Verfahren zur Musterfindung in Texten und sind aus dem Alltag nicht mehr wegzudenken.

2 Algorithmen

2.1 Naive Suche

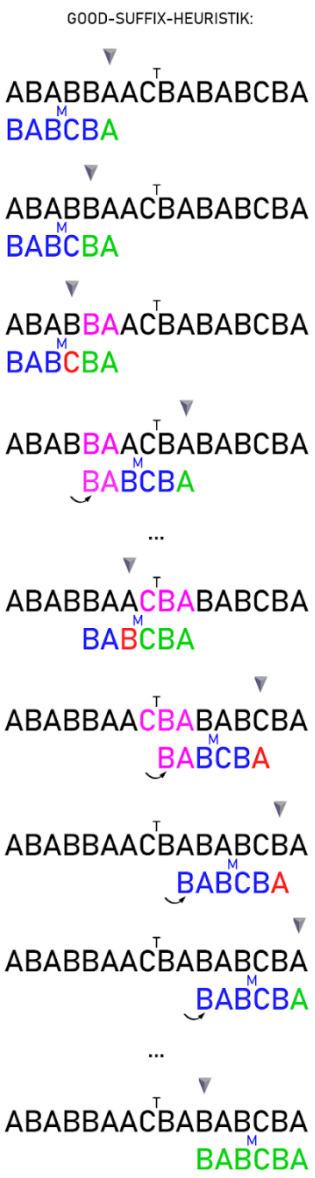
Die naive Suche beschreibt einen intuitiven, wenn auch ineffizienten Algorithmus zur Findung von Zeichenketten in Texten, welcher heutzutage aufgrund seiner Geschwindigkeit keine wirkliche Anwendung mehr hat. Initial wird die Suchmaske (M) an Position 0 des Textes (T) angelegt und folgend von links nach rechts jedes Zeichen von M mit dem dazugehörigen Zeichen aus T verglichen. Tritt eine Nichtübereinstimmung (Mismatch) auf, wird M um ein Zeichen nach rechts in T verschoben und wieder ab dem ersten Zeichen von M verglichen. Dieser Ansatz bedarf keiner Vorbereitungszeit, ist aber aufgrund vieler redundanter Vergleiche mit einer Suchzeit von O((n - m + 1) * m) langsam. Hierbei beschreiben n die Länge von T und m die Länge von M.

2.2 Boyer Moore



Dieser 1977 entwickelte Algorithmus eignet sich besonders im Einsatz mit langen Suchmasken. Insbesondere kann er sublinear sein, da nicht jedes Zeichen von T gelesen werden muss. Im Gegensatz zur naiven Suche werden die Zeichen von M und T nicht von links nach rechts in M, sondern von rechts nach links verglichen, weshalb zunächst mit einer Vorbereitungszeit von O(m) gerechnet werden muss. Kommt es zu einem Mismatch, wird eine Fallunterscheidung durchgeführt und eine von zwei Heuristiken angewendet. Die Bad-Character-Heuristik wird eingesetzt, wenn beim Vergleich des ersten Elements von M und T ein Bad-Character, also ein nicht übereinstimmendes Zeichen, auftritt. In M wird nach dem nächsten Vorkommen dieses Zeichens aus T gesucht und die Maske dann so weit nach rechts verschoben, bis beide Zeichen übereinanderliegen. Existiert das Bad-Character aus T nicht in M, dann wird M um seine volle Länge nach rechts verschoben.

Die Good-Suffix-Heuristik findet Verwendung, wenn an aktueller Position ein Suffix von M mit dem dazugehörenden Suffix von T gleicher Länge übereinstimmt und danach ein Mismatch auftritt. In diesem Fall wird M so weit verschoben, bis die nächste Wiederholung des Suffixes in M auf dem Suffix in T liegt. Wiederholt sich das Suffix nicht in M, dann wird M um seine volle Länge nach rechts verschoben. Die Suchzeit kann von O(n/m) im Idealfall bis zu O(n*m) im schlechtesten Fall reichen.



2.3 Knuth-Morris-Pratt

Der Algorithmus von Knuth, Morris und Pratt “KMP 77” baut auf der naiven Suche auf, ist allerdings deutlich performanter, da M um mehr als eine Position nach rechts verschoben werden kann. Im Gegensatz zum naiven Ansatz und der Lösung von Boyer und Moore besteht der Algorithmus aus zwei Teilen - einer Präfix-Analyse (Preprocessing) und der Suche.

Während des Preprocessing analysiert der Algorithmus zunächst M und speichert Informationen über seine Struktur in einem Array. Aufgabe ist es, für jedes Präfix von M die Länge seines breitesten Randes zu bestimmen. Unter einem Rand versteht man ein Wort, welches echtes Präfix und Suffix von M, also in welchem das Präfix gleich dem Suffix ist. Die Vorbereitungszeit beträgt O(m). Die Suchphase verhält sich zu Beginn gleich der naiven Suche. Kommt es aber zu einem Mismatch, wird die neue Position von M mit folgender Formel berechnet:

aktuelle Position von M in T **+** Länge des übereinstimmenden Präfixes **-** Wert aus Array an Stelle: [Länge des übereinstimmenden Präfixes - 1]

Bei einem Mismatch an Stelle 0 von M, wird M stets eins nach rechts verschoben. Zu beachten ist, dass nach einer Verschiebung von M mit dem Vergleich direkt an der Position des Mismatch in T fortgefahren wird, wenn die beiden Präfixe von M und T ab der Position von M wieder übereinstimmen. Diese Übereinstimmung lässt sich aus dem zuvor generierten Array erkennen. Der KMP Algorithmus funktioniert besonders gut mit kleinen Alphabeten, da die Wahrscheinlichkeit von mehrmals vorkommenden Präfixen hoch ist. Die Suchzeit beträgt O(n).

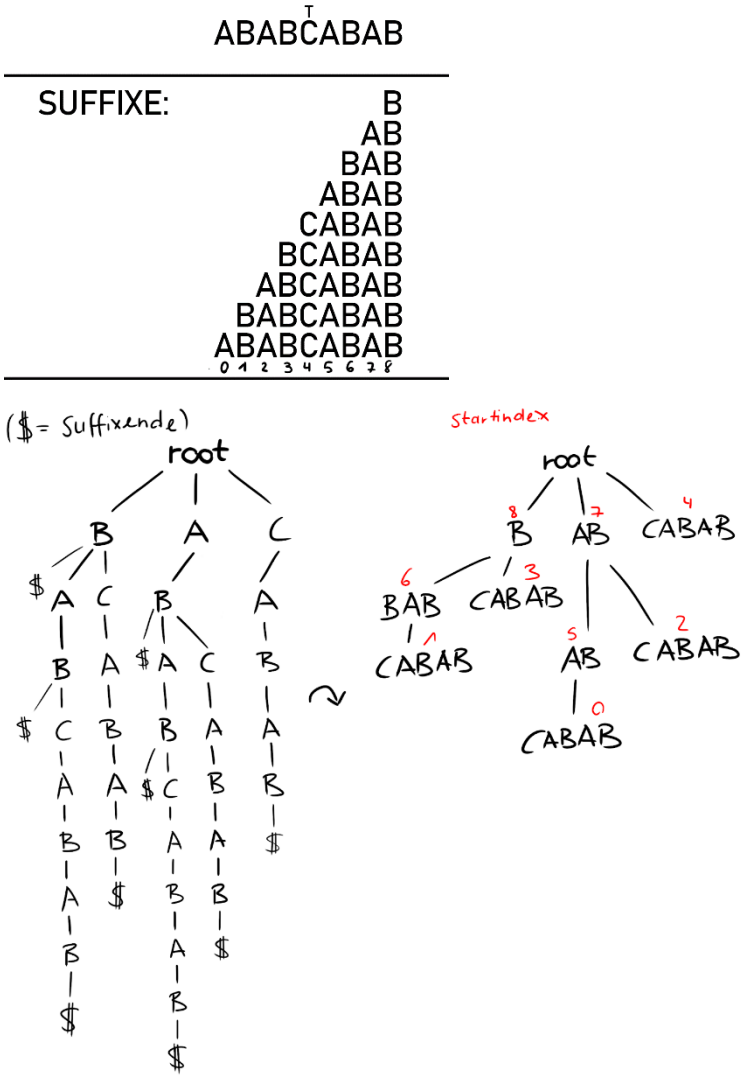
										Länge
Position:	0	1	2	3	4	5	6	7	8	
Muster:	a	b	a	b	c	a	b	a	b	**9**
Präfix (0..0):										**0**
Präfix (0..1):										**0**
Präfix (0..2):	*a*		a							**1**
Präfix (0..3):	*a*	*b*	a	b						**2**
Präfix (0..4):										**0**
Präfix (0..5):	*a*					a				**1**
Präfix (0..6):	*a*	*b*				a	b			**2**
Präfix (0..7):	*a*	*b*	*a*			a	b	a		**3**
Präfix (0..8):	*a*	*b*	*a*	*b*		a	b	a	b	**4**

Position:	*0*	*1*	*2*	*3*	*4*	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	...
Text:	*a*	*b*	*a*	*b*	**a**	b	c	b	a	b	a	b	c	a	b	a	b	c	a	b	...
Muster:	*a*	*b*	*a*	*b*	**c**	a	b	a	b												
Position:	0	1	2	3	*4*	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	...
Text:	a	b	*a*	*b*	a	b	c	b	a	b	a	b	c	a	b	a	b	c	a	b	...
Muster:			*a*	*b*	a	b	c	a	b	a	b										
Position:	0	1	2	3	*4*	*5*	*6*	*7*	8	9	10	11	12	13	14	15	16	17	18	19	...
Text:	a	b	a	b	*a*	*b*	*c*	**b**	a	b	a	b	c	a	b	a	b	c	a	b	...
Muster:			a	b	*a*	*b*	*c*	**a**	b	a	b										
Position:	0	1	2	3	4	5	6	*7*	8	9	10	11	12	13	14	15	16	17	18	19	...
Text:	a	b	a	b	a	b	c	b	a	b	a	b	c	a	b	a	b	c	a	b	...
Muster:								a	b	a	b	c	a	b	a	b					
Position:	0	1	2	3	4	5	6	*7*	8	9	10	11	12	13	14	15	16	17	18	19	...
Text:	a	b	a	b	a	b	c	**b**	a	b	a	b	c	a	b	a	b	c	a	b	...
Muster:								**a**	b	a	b	c	a	b	a	b					
Position:	0	1	2	3	4	5	6	7	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*	*16*	17	18	19	...
Text:	a	b	a	b	a	b	c	b	*a*	*b*	*a*	*b*	*c*	*a*	*b*	*a*	*b*	c	a	b	...
Muster:									*a*	*b*	*a*	*b*	*c*	*a*	*b*	*a*	*b*				
Position:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	*17*	*18*	*19*	...
Text:	a	b	a	b	a	b	c	b	a	b	a	b	c	*a*	*b*	*a*	*b*	*c*	*a*	*b*	...
Muster:														*a*	*b*	*a*	*b*	*c*	*a*	*b*	...

Bildquelle: https://de.wikipedia.org/wiki/Knuth-Morris-Pratt-Algorithmus (25.10.2022)

2.4 Suffix-Baum

Unter Umständen kann die Textsuche mit einem Suffix-Baum noch effizienter gestaltet werden. Während des Preprocessing wird zunächst der Suchbaum aus allen Suffixen von T generiert. Dabei wird mit dem Suffix maximaler Länge begonnen und sich der Reihenfolge nach bis zum kürzesten Suffix mit Länge 1 vorgearbeitet. Für jedes dieser Suffixe werden nach selbem Prinzip durch Rekursion alle Substrings gebildet. Dabei wird für jeden Substring überprüft, ob es für das erste Zeichen bereits eine entsprechende Kante gibt. Falls nicht, wird diese erstellt. Dieser Prozess nimmt O(n) Zeit in Anspruch, wobei n die Länge von T ist.
In der fertigen Baumstruktur gibt es dann für jedes Wort nur einen einzigen Pfad. Dies macht man sich bei der Suche zunutze, da für M ein bestimmter Pfad von Knoten zu Knoten verfolgt wird und man mit Sicherheit sagen kann, dass ein Wort nicht in T existiert, wenn es keine weitere Verbindung mehr zu einem Knoten mit den benötigten Substring aus M gibt. Dies resultiert in einer geringen Suchzeit von O(m), wobei m die Länge von M ist.



3 Didaktisches Konzept

3.1 Zielgruppe

Eine der grundlegenden Fähigkeiten eines Informatikers ist die Entwicklung und Anwendung von Algorithmen zur effizienten Lösung von Problemen im Alltag. Dabei stellt die Zeichenkettensuche in Texten heute eines der bedeutendsten Probleme dar. Für die Arbeit mit Texteditoren, Literaturdatenbanken, Bibliothekssystemen und vielen weiteren Anwendungen, welche eine schnelle Suchfunktion voraussetzen, ist die Lösung dieses Problems erforderlich. Aus diesem Grund sollte insbesondere angehenden Informatikerinnen und Informatikern ein Einblick in dieses Themengebiet gegeben werden.

3.2 Lehr- und Lernziele

Ziel ist es, ein Verständnis für einige der heute relevantesten Suchalgorithmen wie der naiven Suche, den Boyer-Moore-Algorithmus, den Knuth-Morris-Pratt-Algorithmus und der Suche mit dem Suffix-Baum zu schaffen. Nach der Simulation ist man in der Lage, die genannten Algorithmen zu verstehen und selbstständig anzuwenden.

3.3 Didaktik

Es gibt eine Einführung in das Thema Zeichenkettensuche in Texten. Die Suchalgorithmen werden einzeln erklärt. Es gibt passende Übungen zu den Themen, wobei man das erlernte Wissen selbst anwenden kann und gleichzeitig noch geführt wird. Fehler werden direkt angezeigt. In der Simulation wird ein Belohnungssystem verwendet.

3.4 Umsetzung

Für diese Aufgabe wurde die geführte Simulation gewählt, da diese den Informatikern über die subjektive Theoriebildung hinaus, mit Praxisnähe, die Komplexität der Algorithmen verständlicher macht.
Im theoretischen Teil der Simulation werden die Algorithmen im Detail erläutert. Diese werden vom Aufbau bis hin zur Anwendung Schritt für Schritt erklärt und auch in Beispielen gezeigt.
Im nachfolgenden praktischen Teil können die Texte als vorgegebenes Beispiel mit dynamischen Suchmustern bearbeitet werden. Dabei führt der Benutzer die Schritte des Algorithmus mit Mausklick und Tasteneingabe aus. Hier erläutern Textboxen die nächsten Schritte so lange, bis diese erfüllt werden.

Beim Starten der Simulation gibt es für jeden Text eine zufällige Auswahl von fünf Suchmustern, sodass die Simulation mehrmals wiederholt werden kann.
Wenn beim Bearbeiten der Beispiele ein Fehler auftritt, wird dies durch ein kurzes Wackeln und rotes Aufleuchten des ausgewählten Symbols angezeigt. In diesem Fall muss der Schritt wiederholt werden, während man eines von drei Leben verliert. Dies soll verhindern, dass die Benutzer durch willkürliches Klicken ans Ziel kommen. Sind alle drei Leben verloren, muss das Beispiel von vorne begonnen werden. Für jeden richtig absolvierten Algorithmus erhält der Benutzer eines von vier Puzzleteilen, welche sich am Ende zu einem mysteriösen Bild zusammensetzen.

3.5 Begründung der Entscheidungen

Der Vorteil dieser Methode ist, dass die vorgegebenen Aufgaben selbst bearbeitet werden und jeder für sich selbst erkennen kann, ob das Thema verstanden wurde. Dies dient der Motivation, da es spannender ist, experimentell zu arbeiten, statt lange Texte zu lesen. Niemand steht unter Zeitdruck, da Fehler keine schwerwiegenden Konsequenzen mit sich führen und man so oft wie man möchte wiederholen kann.

4 Visuelles Konzept

4.1 Designentscheidungen zu Visualisierung und Interaktion

Die generelle visuelle Thematik der geführten Simulation ist ein altertümlicher Schreibtisch, an dem noch mit Tusche und Feder geschrieben wird. Der zu durchsuchende Text ist ein Ausschnitt eines Buches, in dem nach einem Suchmuster, welches auf einem Notizzettel steht, gesucht wird. Da es dunkel ist, braucht der Nutzer eine bewegbare Kerze, um einen Teil des Textes zu sehen.

Am oberen Bildschirmrand befindet sich ein Text mit Anweisungen, welcher dem Nutzer sagt, was er als Nächstes zu tun hat. Über jedem Zeichen im Text und Suchmuster wird der entsprechende Index angezeigt. An der aktuell betrachteten Stelle im Text befindet sich ein Pointer in Form eines Pfeils nach unten.

Das Suchmuster ist direkt unter der Zeile im Text platziert, die gerade betrachtet wird. Es kann mittels Maus nach links und rechts bewegt werden, so wie der Algorithmus vorgehen würde. Beim Vergleichen der Zeichen kann man mit Tasteneingabe “stimmt überein” oder “stimmt nicht überein” an das System übergeben. Wenn diese Entscheidung richtig ist, geht der Vorgang problemlos weiter. Bei fehlerhafter Eingabe färbt sich der Bildschirm kurz rot und der Pointer bleibt nach einem kurzen Rütteln an seinem Platz.
Wenn der Nutzer eine Übereinstimmung mit “stimmt überein” korrekt eingibt, springt der Pointer um eine Stelle weiter und das entsprechende Zeichen wird grün hinterlegt. Sobald man einen Unterschied mit der Eingabe “stimmt nicht überein” richtig erkennt, erscheint kurz der Schriftzug “Mismatch” und das Textfeld zeigt den nächsten Schritt.
Diese Vorgehensweise betrifft alle Algorithmen außer den Suffix-Baum-Algorithmus.

Beim Suffix-Baum-Algorithmus stellt der Nutzer zuerst den Baum zusammen. Danach kann durch die Ebenen der Baumstruktur mit Tasteneingabe navigiert werden, welches visuell durch eine zeigende Hand dargestellt wird.
Am Ende des Baumes findet man den Index der gesuchten Stelle im Text, welche rot umrandet wird. Diese muss im Text schlussendlich gefunden werden.

Der Nutzer hat für jede Simulation eine gewisse Anzahl an Lebenspunkten, wobei bei jedem Fehler ein Leben abgezogen wird. Wenn null Leben erreicht werden, muss der jeweilige Algorithmus von neuem gestartet werden. Bei jedem Neustart der Seite wird ein zufälliges Beispiel ausgewählt.
Wenn eine Simulation erfolgreich abgeschlossen wurde, erhält der Nutzer ein Puzzleteil. Wenn man alle vier Puzzleteile erhalten hat, setzen sich diese zu einem mysteriösen Bild zusammen.

4.2 Begründung der Entscheidungen

Die generelle visuelle Thematik wurde gewählt, da sie atmosphärisch angenehm ist und zum Thema Zeichenketten gut passt, da es um Zeichen/Schrift geht. Durch das allgemein dunkle Erscheinungsbild der Simulation, gleicht es einem “Dark Mode”, welcher auch beim nächtlichen Lernen angenehm für die Augen ist.

Durch die wechselnden Anweisungen muss sich der Nutzer immer nur mit einem Schritt beschäftigen und wird nicht von Anfang an mit der gesamten Komplexität des Algorithmus überfordert und bleibt so motiviert.
Die angezeigten Indizes und der “Pointer” dienen der besseren Übersichtlichkeit und sind unumgänglich für den Algorithmus.

Das Suchmuster wird direkt unter dem Text platziert, damit die Vergleiche leicht ersichtlich sind und der Nutzer mit seinem Blick nicht immer über den ganzen Bildschirm wandern muss.
Mit dem manuellen Verschieben des Suchmusters mit der Maus sollte dieser Schritt besser im Gedächtnis des Nutzers bleiben. Wenn dieser Vorgang automatisch wäre, würde man nicht herausfinden müssen, nach welchem Prinzip die Verschiebung stattfindet. So prüft man, ob der Nutzer diese Vorgänge verstanden hat. Gleiches gilt für die Tasteneingaben.
Durch die optischen Hervorhebungen einer fehlerhaften Eingabe, soll dem Nutzer dieser Fehler sofort auffallen und zum Korrigieren anregen. Die grüne Hinterlegung soll ein kleines Erfolgserlebnis hervorrufen und zum Weitermachen motivieren. Das Einblenden des Schriftzuges “Mismatch”, wenn jenes gefunden wird, soll deutlich machen, dass der Nutzer keinen Fehler gemacht hat, sondern lediglich einen Fehler gefunden hat. Bei Rotfärbung hätte es zu Missverständnissen diesbezüglich kommen können.

Die Einführung der Lebenspunkte soll den Nutzer daran hindern, willkürliche Eingaben zu machen, bis die richtige entdeckt wird. Dadurch, dass man von Neuem starten muss, soll man mit Verständnis weiterkommen können, als durch stumpfes ausprobieren.

Das Belohnungssystem mit den Puzzleteilen dient der Motivation des Nutzers, alle Simulationen der verschiedenen Algorithmen zu durchlaufen. Da man weiß, dass man etwas erhält, aber nicht genau was, wird die Neugier angeregt.

5 Verweise

5.1 Einführung zum Thema

https://www.geeksforgeeks.org/boyer-moore-algorithm-for-pattern-searching/ (11.10.2022)

https://www.geeksforgeeks.org/kmp-algorithm-for-pattern-searching/ (11.10.2022)

https://www.inf.hs-flensburg.de/lang/algorithmen/pattern/kmp.htm (12.10.2022)

https://www.inf.hs-flensburg.de/lang/algorithmen/pattern/bm.htm (12.10.2022)

https://www.geeksforgeeks.org/pattern-searching-using-trie-suffixes/ (12.10.2022)

https://www.uni-ulm.de/fileadmin/website_uni_ulm/iui.inst.190/Mitarbeiter/ohlebusch/PAPERS/SPIRE02.pdf (16.10.2022)

5.2 Didaktisches Konzept

https://www.grin.com/document/32855 (15.10.2022)